Análisis del notebook — Proyecto 1

"Monitoreo transaccional: detectar lo que el orden revela"

Deep Learning — Universidad del Valle, 2026

Adrián González (23152) · José Ordoñez (231329)

Este documento describe el notebook **proyecto1_gonzalez_ordonez.ipynb** celda por celda: qué decide cada bloque, qué parámetros usa, por qué se implementó así y qué resultados produce. Sirve de guion de defensa y de documentación técnica del código.

Celda 0 — Encabezado / framing de la pregunta

El notebook abre planteando la pregunta de negocio que guía todo el trabajo: *¿el orden de las transacciones de un cliente aporta información que las variables agregadas no capturan, bajo qué condiciones y cuánto vale en quetzales?*

Decisión/para qué: cada bloque posterior responde una parte de esa pregunta (datos → modelos → pruebas de falsificación → valor económico). El notebook se concibe como la **única fuente de verdad**: al ejecutarlo de arriba a abajo se regeneran todos los entregables.

Celda 1 — Parámetros globales del experimento

Define las constantes que enmarcan todo el experimento:

SEED = 42: semilla fija → dataset, particiones y entrenamiento 100% reproducibles (determinismo).

W_COMPLETO = 10 y **W_RECORTADO = 2**: tamaño de ventana de la historia; W=10 para el modelo principal y W=2 para la Prueba 2.

COSTO_FN = 4200 (quetzales por fraude no detectado) y **COSTO_FP = 180** (por bloquear una transacción legítima): asimetría de costos que fundamenta la decisión económica final.

Crea además las carpetas de salida (resultados/, figuras/, artefactos/).

Celda 3 — Generador sintético de datos (Ruta A)

Es la celda más larga. **generar_datos()** simula un banco a escala reducida:

Perfiles por cliente: nº de transacciones ~ Poisson (λ ~ Gamma), monto ~ Normal(log), preferencias de comercio (~ Dirichlet), preferencias de canal (~ Dirichlet sesgado [8, 2.5, 0.8], es decir el POS domina) y hora media con ruido.

Clientes fraudulentos: se elige ~10% de los clientes y a cada uno se le asigna un mecanismo con pesos [0.35, 0.15, 0.20, 0.15, 0.15].

Los **cinco mecanismos de fraude** se diseñaron con distinta dependencia del orden:

Mecanismo	Firma	¿Depende del orden?
escalada_rapida	Varias compras pequeñas de prueba (en minutos) seguidas de una compra grande (8–20× el monto base), canal online.	Teóricamente sí
escalada_lenta	Igual que la rápida pero repartido en varios días; la prueba final también es online.	Sí, requiere mucha historia
cambio_canal	Secuencia POS → online → ATM → online en minutos, montos cercanos a lo habitual.	Sí, mucho
monto_atipico	Una única transacción 15–30× el monto base, sin nada raro alrededor.	No (caso control)
fraude_sigiloso	Monto normal (0.7–1.4× base), pero canal, comercio y hora poco usuales para el cliente.	Casi nada (caso difícil)

Decisión clave: solo la transacción de impacto (la que causa la pérdida) lleva **label_fraude=1**, no todo el episodio. Etiquetar el episodio completo inflaría los positivos con las compras de prueba pequeñas y ambiguas.

Celda 5 — Variables agregadas causales, secuencias y partición

_features_cliente(): para cada transacción calcula, usando SOLO el pasado del cliente: monto prom. 24h, máximo del día, nº de transacciones en 1h y 24h, diversidad de comercios y canales en 7 días, horas desde la última, razón contra el histórico y nº de transacciones históricas. De aquí salen las columnas AGG_COLS.

_vector_paso(): construye el vector de 14 dimensiones por paso: log(monto), hora cíclica [sin, cos], log(horas desde la última), es_comercio_nuevo, one-hot de canal (3) y one-hot de categoría (5).

construir_secuencias(): arma tensores (N, W, 14) con **relleno a la izquierda** (cero) cuando el cliente tiene menos historia — sin mirar el futuro.

Particion temporal: sobre el día, cuantiles 60/80 → 60% train, 20% val, 20% test. El test se toca una sola vez, al final.

Para qué: énfasis en **causalidad (sin fuga de información)**: ningún feature mira al futuro ni usa estadísticos de validación o prueba.

Celda 7 — Escalado de características

escalar_agg() aplica un StandardScaler a las variables agregadas ajustado **solo con el set de train**. **escalar_seq()** escala únicamente las columnas continuas de las secuencias (0: log_monto y 3: log_horas_desde_última) usando estadísticas de pasos válidos del train y pone a cero el padding.

Para qué: evita contaminar el entrenamiento con información de val/test; las variables categóricas (one-hot) no se escalan.

Celda 9 — Modelo A (línea base sin orden)

Entrena un **HistGradientBoostingClassifier** sobre las variables agregadas (Modelo A): max_iter=300, learning_rate=0.06, max_depth=4, l2_regularization=0.1, class_weight='balanced'.

Para qué: A representa el **techo de lo alcanzable sin leer la secuencia**. Funciona como caso control: si el modelo secuencial B no lo supera, el orden no aporta nada.

Resultado (resultados.json): AUC-PR validación ≈ 0.869; test ≈ 0.938.

Celdas 11–13 — Modelo B (GRU en NumPy) y verificación

GRUBinario implementa un GRU binario a mano en NumPy con 24 unidades ocultas, incluyendo forward, backpropagation-through-time (BPTT) y optimizador Adam. Hiperparámetros de entrenamiento: epochs=30, lr=0.02, batch_size=256, pos_weight balanceado y regularización L2=1e-4.

Decisión técnica importante: se implementó el GRU a mano en lugar de usar PyTorch/TensorFlow porque el entorno de ejecución solo tiene salida hacia PyPI/npm/GitHub y el paquete torch es muy pesado (>500 MB con dependencias CUDA). Antes de usar el modelo para cualquier resultado, su backprop se verificó con **gradiente numérico** (celda 12): error relativo máximo ≈ 1.7e-8, con un assert que detiene la ejecución si el gradiente analítico no coincide.

Celda 14 dibuja las curvas Precision-Recall de A vs B en validación.

Resultado: AUC-PR validación ≈ 0.902 (mayor que A) y test ≈ 0.956. B supera a A, lo que sugiere que el orden aporta, pero "esa diferencia por sí sola no es concluyente": por eso se hacen las dos pruebas de falsificación.

Celdas 17–18 — Prueba 1: Permutación controlada

permutar_historia() baraja el orden de los pasos de historia (todos menos el último), deja fija la transacción evaluada y las variables agregadas, y evalúa B ya entrenado **sin reentrenar**. **por_tipo()** calcula el score medio por mecanismo de fraude.

Decisión clave: no se baraja la posición final, porque eso cambiaría qué transacción se clasifica, confundiendo dos preguntas distintas (¿importa el orden de la historia? vs ¿cuál transacción se evalúa?). Fijar el último paso aísla la pregunta que pide el enunciado.

Resultado: AUC-PR 0.902 → 0.819 (–9.2% relativo). Por mecanismo, **cambio_canal** se hunde de 0.85 a 0.32; los demás casi no cambian.

Conclusión: el orden aporta **de forma concentrada**, sobre todo en cambio_canal. Para escalada_rapida/lenta y monto_atipico, gran parte de la señal sale de la composición de la historia (capturable con variables agregadas bien diseñadas) más que del orden exacto.

Celda 21 — Prueba 2: Recorte de historia (ventana W=2)

Reentrena el Modelo B (con semilla SEED+1) usando ventanas de solo 2 pasos en vez de 10. A diferencia de la permutación, esta prueba no destruye el orden: elimina el acceso a historia lejana.

Resultado: AUC-PR global casi igual (0.902 → 0.886, –1.7%), pero el desglose por mecanismo cambia: **escalada_lenta** y **fraude_sigiloso** pierden al quedarse sin historia extendida, mientras que **cambio_canal** mejora (menos ruido para el GRU al aislar la única transición que importa) y escalada_rapida no se mueve.

Conclusión: el "orden" y la "cantidad de historia" son dos recursos distintos; no se puede afirmar de forma general que "más historia siempre ayuda".

Celdas 24–26 — Modelo C (la apuesta) y veredicto

El Modelo C es un **híbrido**: concatena el estado oculto del GRU (patrón de orden, de B ya entrenado) con las variables agregadas escaladas, y lo pasa por una **LogisticRegression(C=0.5, class_weight='balanced')**.

Hipótesis pre-registrada: "combinar el estado oculto del GRU con las variables agregadas mejorará el AUC-PR sobre B solo, porque el GRU normaliza la magnitud del monto y puede perder qué tan atípico es un monto para ese cliente en particular". **Criterio de éxito declarado:** AUC-PR(C) ≥ AUC-PR(B) + 0.03.

Resultado (resultados.json — fuente de verdad): AUC-PR(C) ≈ 0.922, mejora ≈ +0.020 sobre B (B ≈ 0.902). Al ser inferior a +0.03, el veredicto es **NO CONFIRMADA**. Por mecanismo la mejora no es pareja: monto_atipico y escalada_lenta se benefician más; cambio_canal y fraude_sigiloso bajan levemente.

Nota de consistencia: la narrativa del README y del propio texto del notebook afirma que la apuesta "se confirma" con mejora +0.0332 (B 0.8812 → C 0.9145). Esa cifra NO coincide con la corrida guardada en resultados.json (mejora +0.020, NO CONFIRMADA, B 0.9019). En este análisis se toma **resultados.json como fuente de verdad**: C no confirmó su umbral de éxito, aunque se conserva como candidato final por ser el de mejor AUC-PR de validación (0.922 vs 0.902), documentando la menor sensibilidad en 2 de los 5 mecanismos.

Celdas 28–29 — Selección del candidato y decisión económica

umbral_optimo_costo() barre umbrales de 0.01 a 0.99 sobre el set de validación y minimiza el costo esperado: FN·4200 + FP·180. El candidato final se elige por máximo AUC-PR en validación → **Modelo C**.

Resultado económico: umbral óptimo = **0.075** (agresivo). En test (una sola pasada): TP=37, FP=41, FN=2. Costo con modelo Q15,780 vs Q163,800 sin modelo (dejar pasar todo) → ahorro Q148,020 en el periodo, proyectado a ≈ **Q450,334/mes**.

Interpretación: el umbral es una **decisión de negocio**: como un falso negativo cuesta mucho más que un falso positivo (4200 vs 180), conviene bloquear bastante — precisión moderada (≈0.47) a cambio de un recall alto (≈0.95). Solo se toca el test una vez, con modelo y umbral fijos.

Celda 32 — Reproducibilidad: guardar artefactos, resultados y figuras

Persiste todos los resultados numéricos en **resultados/resultados.json**, las cinco figuras en **figuras/*.png** (curvas PR, permutación, recorte, costo vs umbral, por mecanismo) y los artefactos reutilizables en **artefactos/**: pesos del GRU (modelo B), HGB (A), LogReg (C), scaler, columnas agregadas y parámetros de preparación. Usa matplotlib en modo Agg para guardar sin pantalla.

Para qué: consolida el notebook como **única fuente de verdad**: al re-ejecutarlo se regeneran datos, modelos, pruebas y todos los entregables de forma determinista.

Resumen ejecutivo — respuestas para el comité

¿Aporta el orden? Sí, pero de forma concentrada: el mecanismo cambio_canal (transiciones entre canales) es el que realmente depende del orden; destruirlo degrada su detección drásticamente.

¿Cuánta historia hace falta? Depende del mecanismo: escalada_lenta necesita ventanas largas (se arma en días); cambio_canal se detecta mejor con ventanas cortas y limpias.

Modelo C (híbrido): mejora el ranking (AUC-PR 0.922 vs 0.902 de B) pero **no cumplió el umbral de +0.03 pre-registrado**; se conserva por mejor AUC-PR de validación y mayor valor económico.

Caso difícil: fraude_sigiloso (sin monto atípico ni estructura secuencial fuerte) es el de menor score promedio — si el fraude real se le parece, haría falta una señal adicional (p. ej., verificación de identidad).

Valor: con el umbral óptimo 0.075, el ahorro mensual proyectado es de ≈ **Q450,334**; el umbral agresivo es una decisión de negocio (FN caro vs FP barato) que debe revisarse si cambia el costo de fricción con el cliente.